

Unidad Didáctica

Aplicaciones dinámicas con Angular,
Unidad III

Contenido

Índice de Contenido	2
Introducción	3
Desarrollo de Contenidos	4
Crear componentes de Angular en aplicaciones en Express.....	4
Configuración de Angular	4
Creación de componentes de Angular	4
Obtención de datos por medio de APIs	5
Validación de formularios	5
Desarrollo de aplicaciones SPA con Angular	6
Configuración del proyecto para SPA.....	6
Cambiando del enrutamiento de Express al enrutamiento de Angular.....	6
Creación las primeras vistas, controladores y servicios	7
Bibliografía y Webgrafía	8
Glosario	8

Introducción



Bienvenidos a la Unidad III de nuestro curso sobre Aplicaciones Dinámicas con Angular. En esta sección, nos adentraremos en el emocionante mundo de Angular y aprenderemos cómo integrarlo en páginas web existentes, acceder a datos a través de APIs, configurar Express para el funcionamiento con aplicaciones de página única (SPA), y aplicar las mejores prácticas en el desarrollo de aplicaciones complejas con esta potente herramienta.

Durante esta unidad, exploraremos cómo crear componentes de Angular en aplicaciones basadas en Express, desde la configuración inicial hasta la obtención de datos mediante APIs y la validación de formularios. También nos sumergiremos en el desarrollo de aplicaciones SPA con Angular, configurando proyectos para este propósito, cambiando del enrutamiento de Express al enrutamiento de Angular, y creando nuestras primeras vistas, controladores y servicios.

Con un enfoque práctico y orientado a resultados, esta unidad nos preparará para construir aplicaciones web dinámicas y eficientes utilizando Angular, aprovechando al máximo su potencial y siguiendo las mejores prácticas en el proceso. ¡Empecemos!

Crear componentes de Angular en aplicaciones en Express

Configuración de Angular

Para configurar Angular en una aplicación Express, primero necesitamos asegurarnos de tener Node.js y npm instalados. Luego, podemos crear un nuevo proyecto de Angular usando Angular CLI:

```
npm install -g @angular/cli  
ng new my-angular-app
```

Una vez que se haya creado el proyecto, podemos integrarlo con nuestra aplicación Express colocando el contenido de la carpeta dist generada por Angular dentro del directorio público de nuestra aplicación Express.

Creación de componentes de Angular

Supongamos que queremos crear un componente de Angular llamado HeaderComponent para mostrar el encabezado de nuestra aplicación. En Angular, podemos crear este componente de la siguiente manera:

```
// header.component.ts  
import { Component } from '@angular/core';  
  
@Component({  
  selector: 'app-header',  
  templateUrl: './header.component.html',  
  styleUrls: ['./header.component.css']  
})  
export class HeaderComponent {  
  // Lógica del componente aquí  
}
```

```
<!-- header.component.html -->  
<header>  
  <h1>¡Bienvenido a nuestra aplicación!</h1>  
</header>
```

Obtención de datos por medio de APIs

Supongamos que queremos obtener datos de una API REST que devuelve una lista de usuarios. En Angular, podemos hacer una solicitud HTTP utilizando el módulo `HttpClient`:

```
import { HttpClient } from '@angular/common/http';

@Injectable()
export class UserService {
  constructor(private http: HttpClient) {}

  getUsers() {
    return this.http.get<User[]>('https://api.example.com/users');
  }
}
```

Validación de formularios

Supongamos que tenemos un formulario de registro de usuarios y queremos validar que el campo de correo electrónico sea válido. Podemos hacerlo utilizando las funciones de validación proporcionadas por Angular:

```
import { FormControl, Validators } from '@angular/forms';

export class RegistrationFormComponent {
  emailFormControl = new FormControl('', [
    Validators.required,
    Validators.email,
  ]);
}
```

```
<input type="email" [formControl]="emailFormControl">
<div *ngIf="emailFormControl.hasError('email') &&
!emailFormControl.hasError('required')">
  Ingresa un correo electrónico válido.
</div>
<div *ngIf="emailFormControl.hasError('required')">
  El correo electrónico es obligatorio.
</div>
```

Estos ejemplos muestran cómo crear componentes, configurar Angular en una aplicación Express, obtener datos de una API y validar formularios en el contexto de una aplicación Angular dentro de un proyecto Express.

Desarrollo de aplicaciones SPA con Angular

En esta sección, exploraremos el desarrollo de aplicaciones de página única (SPA) utilizando Angular. Las SPAs ofrecen una experiencia de usuario fluida al cargar una sola página web que se actualiza dinámicamente según las acciones del usuario, en lugar de cargar páginas completamente nuevas desde el servidor.

Configuración del proyecto para SPA

Para configurar un proyecto Angular como una SPA, primero necesitamos asegurarnos de tener Angular CLI instalado. Luego, podemos crear un nuevo proyecto Angular y configurarlo para ser una SPA utilizando Angular CLI de la siguiente manera: `ng new my-spa-app --routing=true`

El parámetro `--routing=true` nos permite generar un archivo de enrutamiento que Angular usará para manejar la navegación dentro de la SPA.

Cambiando del enrutamiento de Express al enrutamiento de Angular

Una vez que tengamos nuestra SPA configurada, necesitaremos cambiar el enrutamiento de Express para que redirija todas las solicitudes a nuestra SPA. Esto significa que todas las rutas definidas en Express deben servir la página principal de la SPA, y luego Angular se encargará de manejar la navegación interna.

Por ejemplo, en Express podríamos configurar una ruta como esta:

```
app.get('*', (req, res) => {  
  res.sendFile(path.join(__dirname, 'public/index.html'));  
});
```

Esta ruta servirá el archivo `index.html` de nuestra SPA para cualquier ruta que no coincida con las rutas definidas específicamente en Express.

Creación las primeras vistas, controladores y servicios

Con nuestra configuración de proyecto y enrutamiento en su lugar, podemos comenzar a crear las vistas, controladores y servicios de nuestra SPA. En Angular, las vistas se definen como componentes, que combinan plantillas HTML con clases TypeScript. Los controladores son las clases TypeScript que definen la lógica de la vista, y los servicios son clases que encapsulan la lógica reutilizable que no está directamente relacionada con una vista específica.

Por ejemplo, podríamos crear un componente de vista llamado HomeComponent, un controlador asociado llamado HomeController, y un servicio llamado DataService para proporcionar datos a la vista.

```
// home.component.ts
import { Component } from '@angular/core';

@Component({
  selector: 'app-home',
  templateUrl: './home.component.html',
  styleUrls: ['./home.component.css']
})
export class HomeComponent {
  // Lógica del componente aquí
}
```

```
<!-- home.component.html -->
<div>
  <h1>Bienvenido a nuestra SPA</h1>
</div>
```

```
// data.service.ts
import { Injectable } from '@angular/core';

@Injectable()
export class DataService {
  getData() {
    return 'Datos de ejemplo';
  }
}
```

Bibliografía y Webgrafía

Gómez, M. Clean JavaScript: Aprende a aplicar Código Limpio, SOLID y Testing. Independently published (10 Abril 2022)

Morales, G. Creando API con Node.js, express y MongoDB. Independently published (31 Agosto 2020)

Azaustre, C. Desarrollo web ágil con Angular.js. Independently published (11 Agosto 2014)

Glosario

- **API** (Interfaz de Programación de Aplicaciones): Conjunto de reglas y protocolos que permite la comunicación entre diferentes componentes de software.
- **Componente**: Elemento fundamental de Angular que encapsula la funcionalidad, la vista y el estilo de una parte específica de la interfaz de usuario.
- **Express**: Framework de desarrollo de aplicaciones web para Node.js que facilita la creación de servidores web y APIs.
- **Enrutamiento**: Proceso de direccionamiento de solicitudes dentro de una aplicación web hacia los recursos adecuados.
- **Formulario**: Elemento de una página web que permite la recolección de datos del usuario.
- **HTML** (Lenguaje de Marcado de Hipertexto): Lenguaje de marcado estándar utilizado para crear páginas web.
- **JavaScript**: Lenguaje de programación ampliamente utilizado en el desarrollo web para crear interactividad en páginas web.
- **Node.js**: Entorno de ejecución de JavaScript basado en el motor V8 de Google Chrome que permite ejecutar JavaScript en el servidor.

- **SPA** (Aplicación de Página Única): Aplicación web que carga dinámicamente una sola página HTML y actualiza la vista de manera dinámica en respuesta a las acciones del usuario.
- **TypeScript**: Superset de JavaScript desarrollado por Microsoft que agrega tipado estático opcional y otras características a JavaScript.
- **Validación de Formularios**: Proceso de verificar la integridad y corrección de los datos ingresados por el usuario en un formulario web.